

Application Architecture

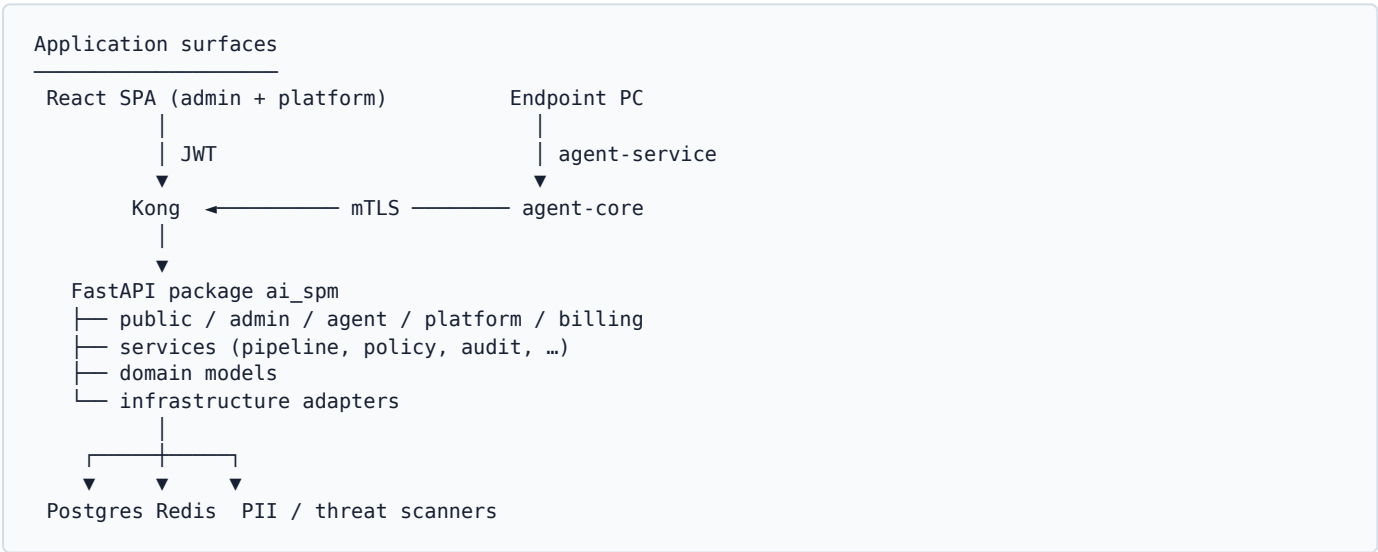
The application shape we follow — layers, technologies, packages, and how console, gateway, and endpoint agent fit together.

Purpose. This document defines the application architecture for AI-SPM: which major components exist, which technologies and packages we use in each layer, and how responsibilities are split. New features should reuse this structure rather than introducing parallel stacks or frameworks.

1. Architecture we follow

AI-SPM is a **hybrid endpoint + central multi-tenant gateway** application:

- 1. **Endpoint agent** (Rust) runs on the managed PC — enrolls, intercepts AI traffic, talks to the gateway over mTLS.
- 2. **Security core** (Python FastAPI) runs policies, PII/threat inspection, audit, quotas, and admin/platform business logic.
- 3. **Admin & platform consoles** (React SPA) are the human UI for company and vendor operators.
- 4. **Edge** (Kong) is the secure front door; **Postgres + Redis** are the durable store and cache.



2. Technology stack (what we use)

LAYER	TECHNOLOGY	ROLE
Backend	Python 3.12+, FastAPI, Pydantic v2, SQLAlchemy 2 async, Alembic, structlog	Security core package <code>ai_spm</code>
Frontend	React 18, TypeScript, Vite 6, React Router 6, TanStack Query 5, Zustand 5, Tailwind 3, Recharts, Lucide	Admin + platform SPA
Database	PostgreSQL 16 + row-level security	Tenant-isolated durable store

LAYER	TECHNOLOGY	ROLE
Cache	Redis 7	Policy cache, signup limits, prompt quota
Background jobs	APScheduler (in-process)	Agent offline detector; no separate broker
Edge gateway	Kong (DB-less)	Rate limit, correlation ID, routing
Auth	bcrypt; tenant JWT; platform JWT; agent mTLS	Separate secrets per plane
PII	Microsoft Presidio + spaCy ([ml] extras); else regex	Detect and mask sensitive values
Threat	Guardrails AI Hub when available; else heuristic patterns	Injection / jailbreak detection
Endpoint agent	Rust (Tokio, hyper, rustls)	MITM, enrollment, heartbeat, local web-audit
Web UI masking	mitmproxy + Python addon	ChatGPT / Claude / Gemini site path
Object storage	MinIO	Installers / artifact stubs
Billing	Stripe webhooks (optional)	Subscription state
Observability	Prometheus + Grafana	Metrics and ops dashboards
Deploy	Docker Compose; Kubernetes HPA; Linux systemd / Windows service	Staging, production, endpoint
CI	GitHub Actions	pytest + ruff, frontend build, cargo test

3. Backend package layout (ai_spm)

All backend code lives under `backend/src/ai_spm/` . We keep a clear ownership split:

PACKAGE / AREA	RESPONSIBILITY
<code>admin/</code>	Company admin HTTP surface
<code>agent/</code>	Endpoint agent HTTP surface (register, heartbeat, prompt, updates)
<code>platform/</code>	Vendor platform surface + provisioning (metadata only)
<code>public/</code>	Unauthenticated onboarding (signup / verify)
<code>billing/</code>	Stripe webhook and subscription helpers
<code>services/</code>	Business orchestration — prompt pipeline, policy, audit, enrollment, dashboard
<code>domain/</code>	SQLAlchemy models and enums
<code>infrastructure/</code>	Adapters: DB, Redis, Presidio, Guardrails, LLM client, auth/password
<code>tenant/</code>	TenantContext, middleware, RLS, quotas
<code>jobs/</code>	APScheduler periodic work
<code>presentation/</code>	Correlation ID, WebSocket helpers

Layering rule we follow

- **Routes** — parse request, auth dependency, call a service, map errors. No business logic.
- **Services** — own the workflow (pipeline order, policy evaluation, audit writes).
- **Domain** — persistence shapes only.
- **Infrastructure** — external systems (scanners, cache, email, DB session). Do not duplicate domain rules here.

Request through the application

Client → Kong → middleware stack → route handler
→ service → adapter / Postgres (RLS) → JSON response

Middleware order (fixed):
CORS → Correlation ID → Quota → Org-id body guard → TenantContext

4. Frontend application structure

AREA	WHAT WE USE IT FOR
<code>src/pages/</code>	Route-level screens (dashboard, agents, policies, audit, platform tenants, marketing/billing as needed)
<code>src/components/</code>	Layout, UI primitives, <code>ProtectedRoute</code>
<code>src/hooks/</code>	TanStack Query data hooks, dashboard WebSocket hook

AREA	WHAT WE USE IT FOR
src/stores/	Zustand auth session (sessionStorage)
src/lib/	Shared client (api.ts), JWT helpers — all HTTP goes through here
src/types/	Shared TypeScript shapes for server responses

Packages we rely on: React 18, React Router, TanStack Query (server state), Zustand (session), Tailwind (styling), Recharts (charts), Lucide (icons), Vite (build).

5. Endpoint agent (Rust workspace)

CRATE / AREA	RESPONSIBILITY
agent-core	Config, gateway client, transparent MITM / proxy, local web-audit, network setup
agent-service	Daemon binary — process lifecycle and startup wiring
agent-installer	Linux guided installer launcher for sealed packages
agent-tray	Tray UI placeholder (status only)

Runtime stack: Tokio async runtime, hyper HTTP, rustls for mTLS. Linux enterprise install uses systemd (ai-spm-agent), transparent redirect for AI traffic, CA trust, and mitmproxy for consumer web UIs. Packaging: sealed Linux .run from Admin Download Agent; Windows path via service + WiX MSI.

6. Security engines (packages)

CAPABILITY	PACKAGE / MODULE	FALLBACK
PII detect / mask	Presidio Analyzer + Anonymizer, spaCy	Shared regex recognizers in-repo
Threat detect	Guardrails AI + Hub validators	Heuristic injection / leakage patterns
Policy	PolicyEngine + Redis cache	—
Prompt lifecycle	PromptPipelineService	Fixed step order; fail-closed on scanner errors

Optional ML extras are installed via backend [ml] extras. Without them the product still boots using regex / pattern fallbacks.

7. Data & tenancy packages

- **ORM / migrations:** SQLAlchemy 2 async + Alembic
- **Driver:** asyncpg
- **Isolation:** JWT org_id → TenantContext → query filters → PostgreSQL RLS (aispm_app / app.current_org_id)
- **Auth helpers:** bcrypt passwords; separate tenant and platform JWT secrets